

Introduction

Dans ce chapitre nous présentons le problème de synchronisation des processus concurrents, et nous définissons les concepts de bases de la programmation concurrente des processus.

1 Problématique

Lorsque deux programmes s'exécutent en même temps, en parallèle, on peut considérer qu'on a deux séquences d'instructions parallèles.

Mais, il est possible que la vitesse de progression des séquences ne soit pas constante dans le temps, et à un moment donné, une séquence peut aller plus vite et en suite ralentir. Plusieurs causes peuvent expliquer un tel changement de rythme. Par exemple, la mémoire virtuelle peut inclure un délai variable selon que le contenu demandé est déjà en mémoire ou non. Il faut se rappeler aussi que le temps d'accès à une information sur disque est variable.

Supposons que plusieurs processus P_i , avec ($i = 0, \dots, N$), incrémentent la valeur de la variable globale M . Le code de ces processus est le suivant :

```
int M = 10 ; // variable globale

void Pi( ) {
    int count = 0 ;
    read(count, M) ;
    count++;
    store(count, M);
}
```

Avec,

- i : le numéro de processus,
- $count$: une variable local utilisée pour incrémenter la valeur de M . Chaque processus a sa propre variable $count$.
- $read(count, M)$: une procédure qui permet de lire la valeur de M et l'affecter a $count$.
- $store(count, M)$: une procédure qui permet de stocker la valeur de $count$ dans M .

A) Supposons que les deux processus P_1 et P_2 s'exécutent séquentiellement (c-a-d, l'un après l'autre).

P ₁	P ₂	Valeur de M
read(count, M)		M= 10
count++		M= 10
store(count, M)		M= 11
	read(count, M)	M= 11
	count++	M= 11
	store(count, M)	M= 12

Initialement M=10, le processus P1 s'exécute jusqu'à la fin est incrément M, alors M = 11. Ensuite, le processus P2 commence son exécution est incrément à son tour la valeur de M. Alors la nouvelle valeur sera M =12.

On remarque que le programme des processus fonction normalement et les résultats des deux exécutions séquentielles sont corrects.

B) Comme les processus sont indépendants, le processeur peut être alloué à chacun des processus dans un ordre quelconque. Mais, les accès concurrents à des données partagées peuvent produire des incohérences de données comme le montre l'exemple suivant :

Supposons maintenant que les deux processus P₁ et P₂ s'exécutent dans l'ordre suivant :

P ₁	P ₂	Valeur de M
read(count, M)		M= 10
	read(count, M)	M= 10
	count++	M= 11
	store(count, M)	M= 11
count++		M= 11
store(count, M)		M= 11

Dans ce scenario, le résultat final est incorrect (M=11 au lieu de M=12). A vrai dire, les processus P₁ et P₂ ne sont pas entièrement indépendants, car ils partagent une ressource commune qui est la variable M.

Donc, si l'accès des processus à cette variable n'est pas contrôlé, cela peut mener inévitablement à des résultats imprévisibles.

Pour éviter ce problème, la variable M doit avoir un seul point d'accès. Dans ce cas, la variable M qui est *partagée* entre les processus est une **ressource critique**. Alors, **un seul processus peut l'accéder à la fois**.

2 Concepts de base

Dans cette section nous présentons quelques définitions des concepts de base pour comprendre le problème de synchronisation des processus concurrents.

2.1 Section Critique (SC)

C'est une partie du code dans un programme (contenant une ou plusieurs instructions) qui accède ou utilise des données communes.

Une section critique dans un ensemble d'instructions implique l'utilisation d'une variable partagée. Mais l'inverse n'est pas toujours vrai (c-a-d l'utilisation d'une variable partagée n'implique pas forcément l'existence d'une section critique).

Lorsque qu'un processus exécute sa section critique, aucun autre processus n'est autorisé à exécuter le code de sa section critique. Alors, l'exécution de la section critique est *mutuellement exclusive*. C'est-à-dire qu'un seul processus peut exécuter la section critique à la fois.

2.2 Ressource Critique (RS)

C'est tout objet (variable, table, fichier, périphérique, ...) qui peut être accédé en concurrence par plusieurs processus. Par exemple deux processus qui tentent d'envoyer chacun, simultanément, un fichier sur l'imprimante. Dans ce cas, l'imprimante est une ressource critique pour les deux processus.

2.3 Exclusion Mutuelle (EM)

Le principe de base d'une solution qui garantit l'exécution simultanée de plusieurs processus sans qu'elle génère des résultats imprévisibles est le suivant:

Avant d'exécuter une SC, un processus doit s'assurer qu'aucun autre processus n'est en cours d'exécution de la même section critique. Dans le cas contraire, il doit attendre que l'autre processus termine sa SC.

// variables globales	
<pre>void *P0(void *arg) { while (true) { <Protocole d'Entrée> <Section Critique> <Protocole de Sortie> <section non-critique> } }</pre>	<pre>void *P1(void *arg) { while (true) { <Protocole d'Entrée> <Section Critique> <Protocole de Sortie> <section non-critique> } }</pre>

Schéma général des algorithmes d'exclusion mutuelle

Pour cela, Il faut ajouter du code dans le programme, juste avant la section critique, et à la sortie de la section critique.

- Un protocole d'entrée : pour vérifier si la SC n'est occupée par aucun autre processus.
- Un protocole de sortie : pour informer (notifier) les autres processus en attente que la SC est devenue libre.

Les protocoles d'entrée et de sortie ne laissent passer qu'un processus à la fois, et il bloque les autres.

2.3.1 Contraintes d'exclusion mutuelle

Toute solution au problème d'exclusion mutuelle doit satisfaire les conditions suivantes :

- **Exclusion Mutuelle** : Si un processus est dans sa section critique, aucun autre processus ne peut y entrer jusqu'à ce qu'il la quitte.
- **Progression** : Lorsqu'un processus n'est pas dans sa section critique, il ne doit pas empêcher les autres processus d'entrer dans leur section critique.
- **Attente finie** : Un processus qui demande l'entrée à sa section critique ne doit pas attendre indéfiniment.
- **Équité** : Tous les processus doivent être égaux vis à vis de l'entrée en SC.

2.4 Instruction atomique

Une instruction est dite *atomique* si son exécution peut être vue comme prenant un temps nul. C'est à dire, si elle est exécutée complètement ou pas du tout, et si son résultat ne peut être affecté par ce qui s'est produit après son début.

Par exemple : $x = x + 1$ c'est une instruction qui incrémente la variable x. Mais en langage machine, elle est traduite par les micro-instructions suivantes :

```
LDA x ;  
ADD 1 ;  
STA x ;
```

Pour que cette l'instruction $x = x + 1$ soit atomique, il faut que leur micro-instructions soient exécuter complètement (sans interruption).